

Curvas de nivel de un terreno

Sebastián Mira P.

Profesora: Nancy Hitschfeld R.

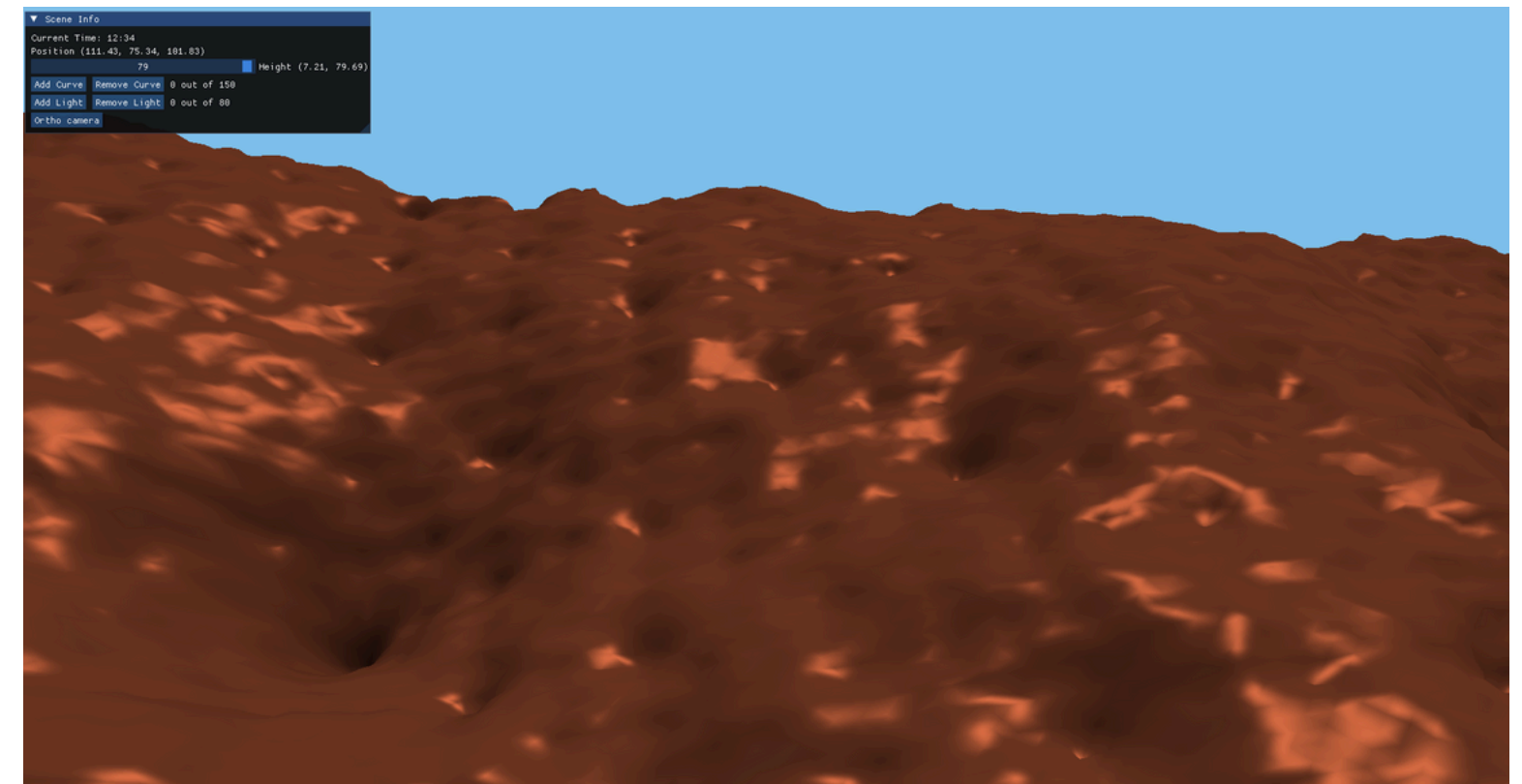
Auxiliar: Vicente I. González

Ayudante: Juan Flores

Contexto

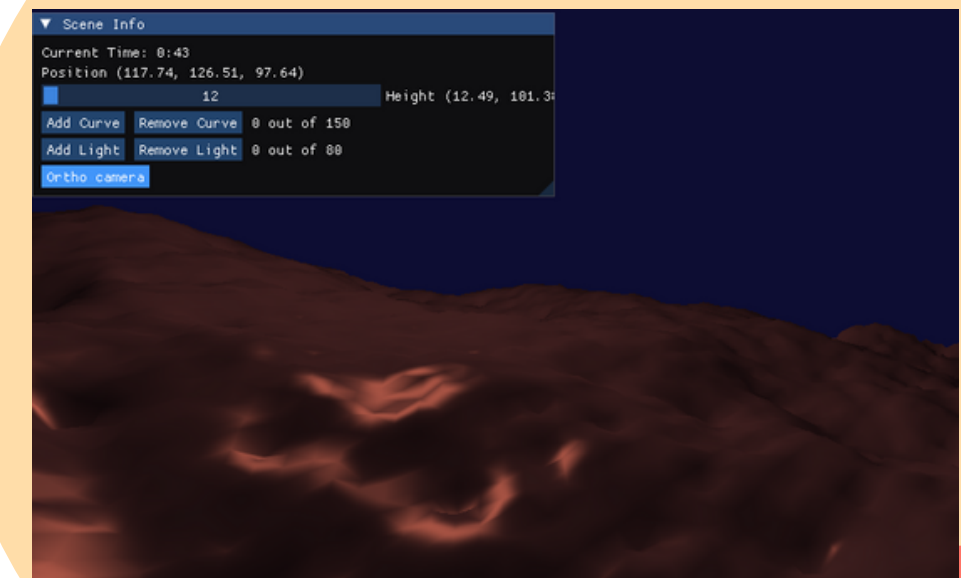
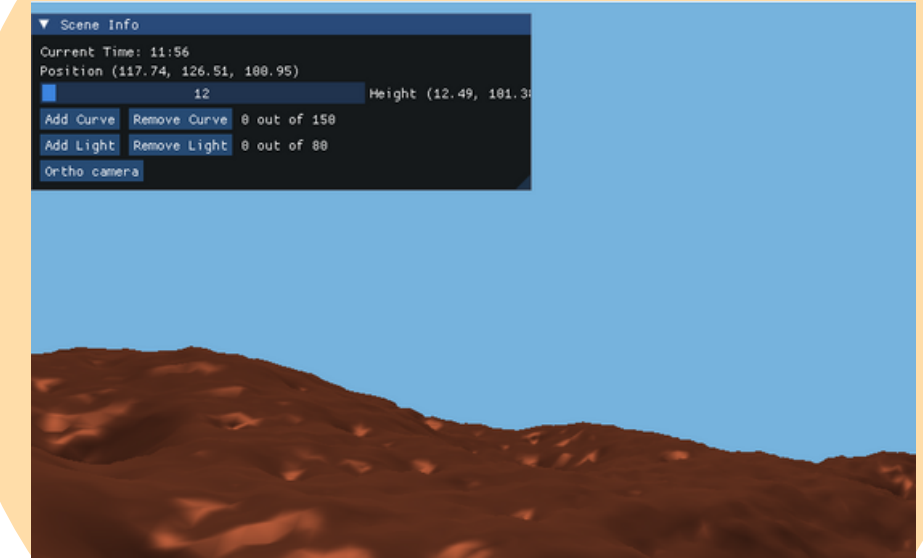
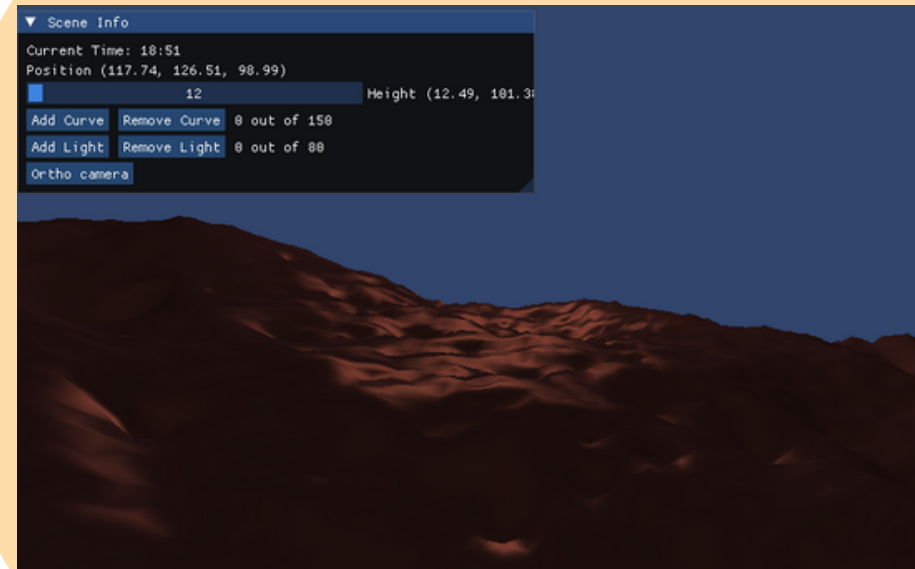
Escena de un terreno generado aleatoriamente en una malla triangular. Cuenta con las siguientes características:

- Iluminado
- Cámara de mov. libre en perspectiva
- Cámara de vista ortogonal
- Interfaz de usuario
- Uso de teclado y mouse



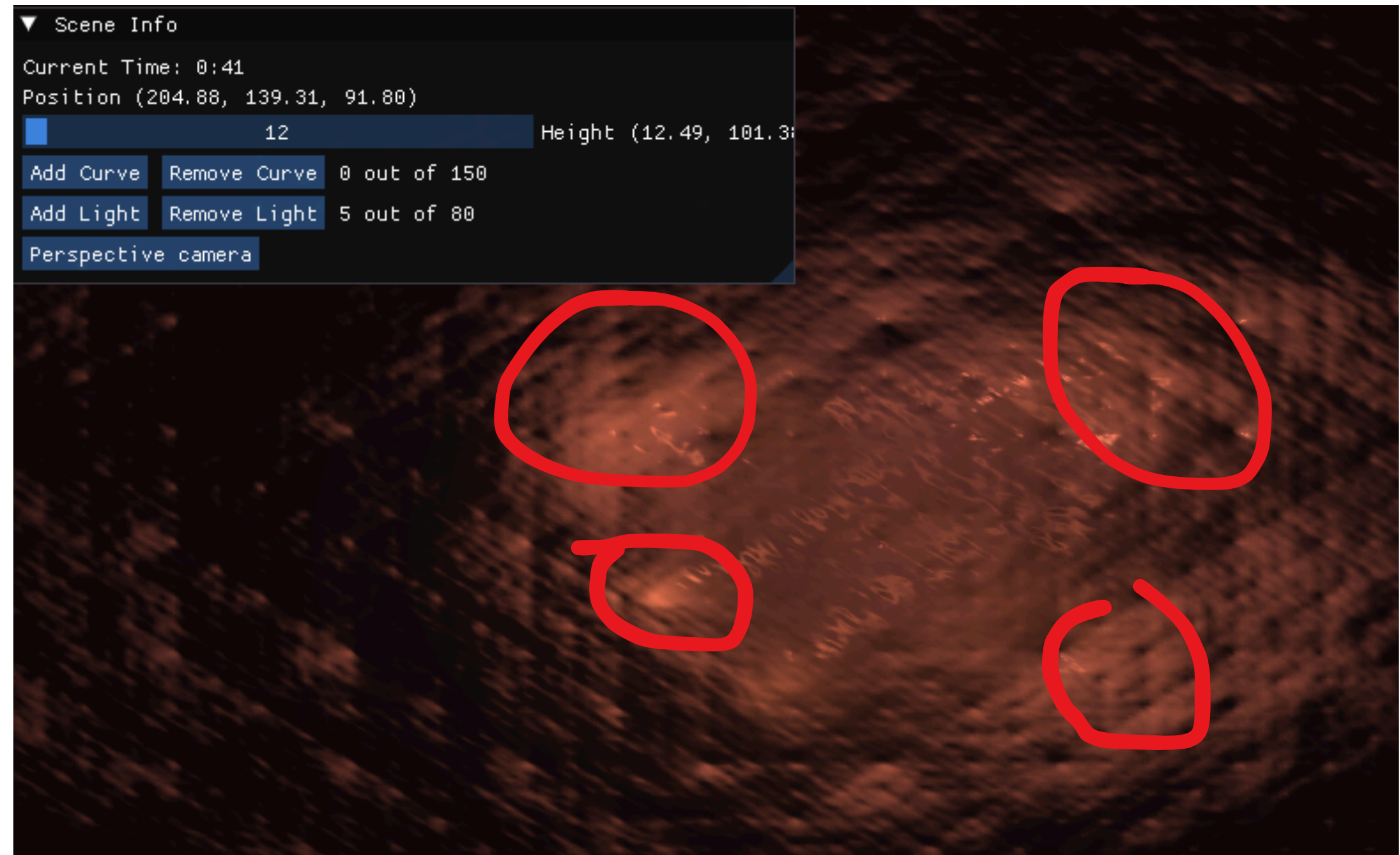
Iluminación

Cuenta con luz ambiental, en todo momento que cambia de color según la hora en la escena. Durante la noche la luna es una luz puntual



Iluminación

Además es posible
colocar nuevas
fuentes pequeñas de
luz puntual



Problemas con Iluminación

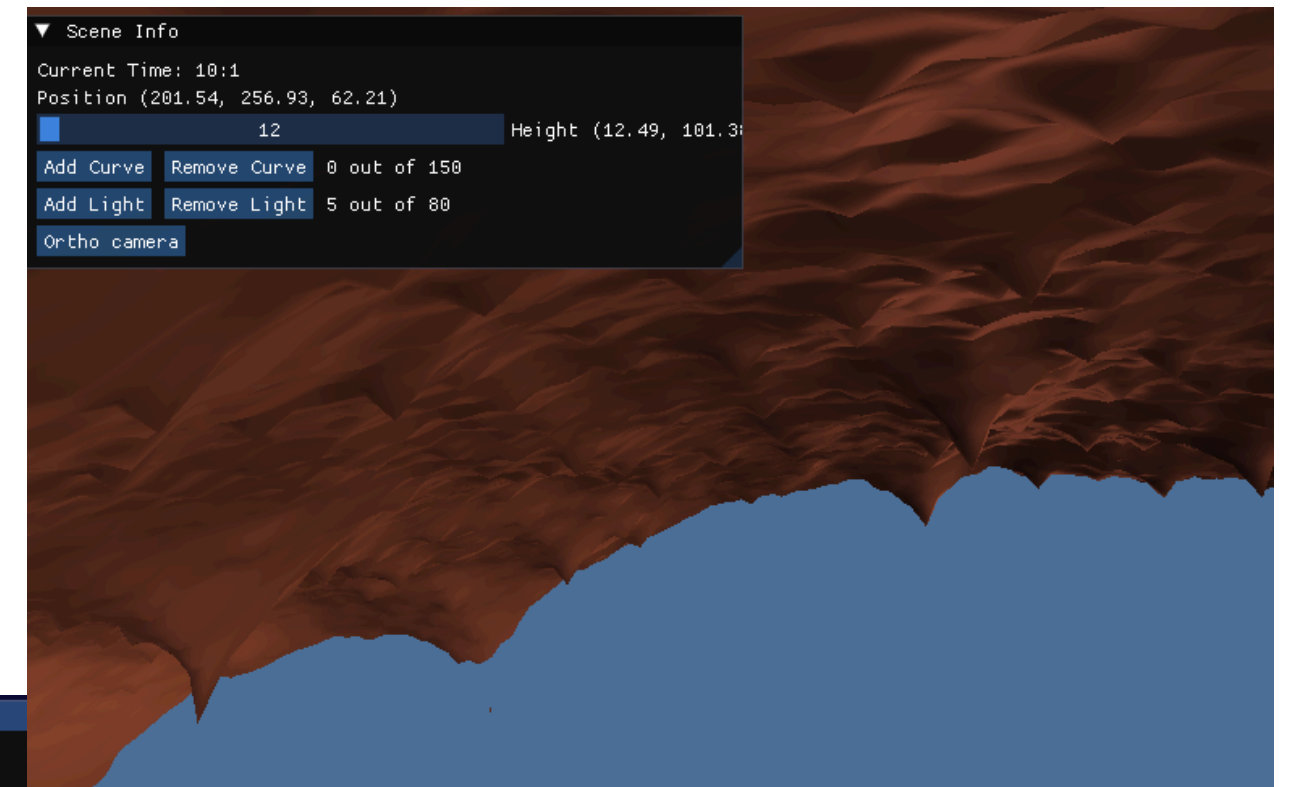
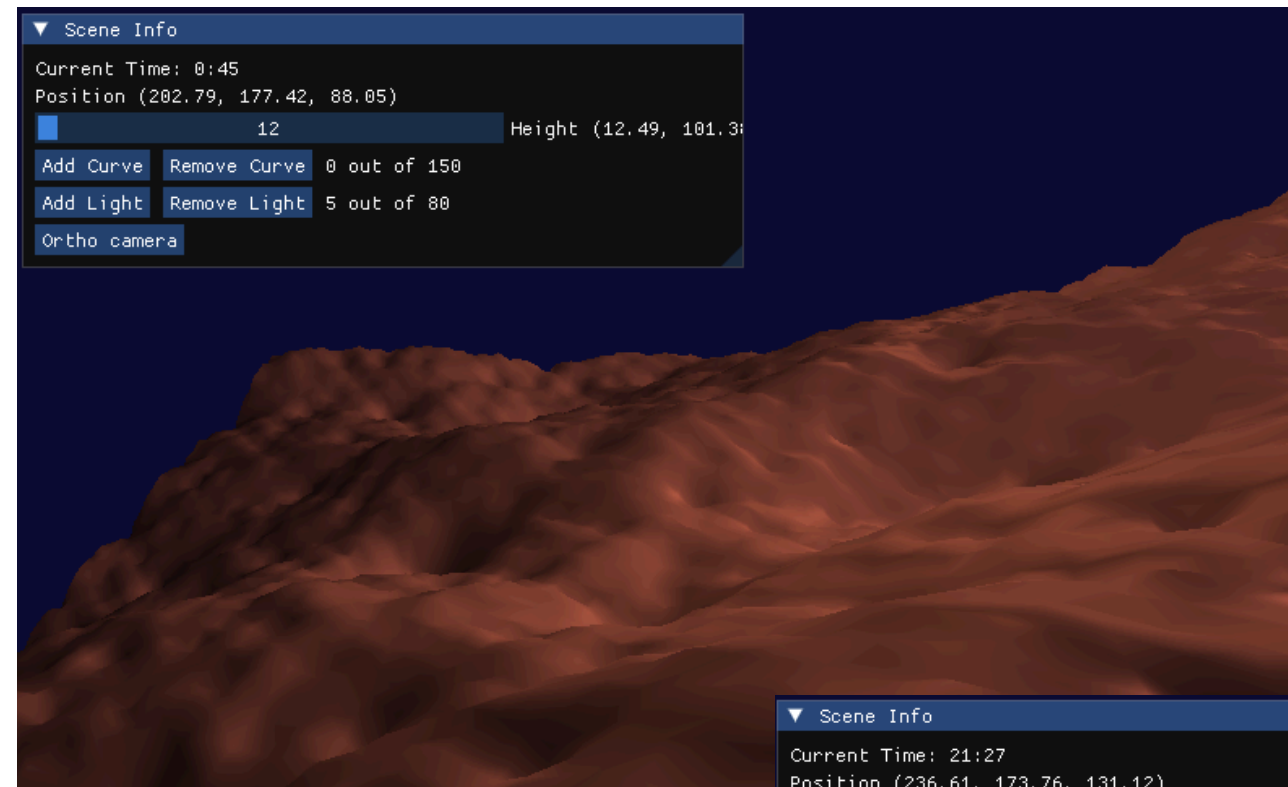
Al dejar la iluminación para el final de la tarea, la inclusión de las normales de los vértices en la información a entregar a los shaders fue complicada (múltiples triángulos usan mismo vértice)



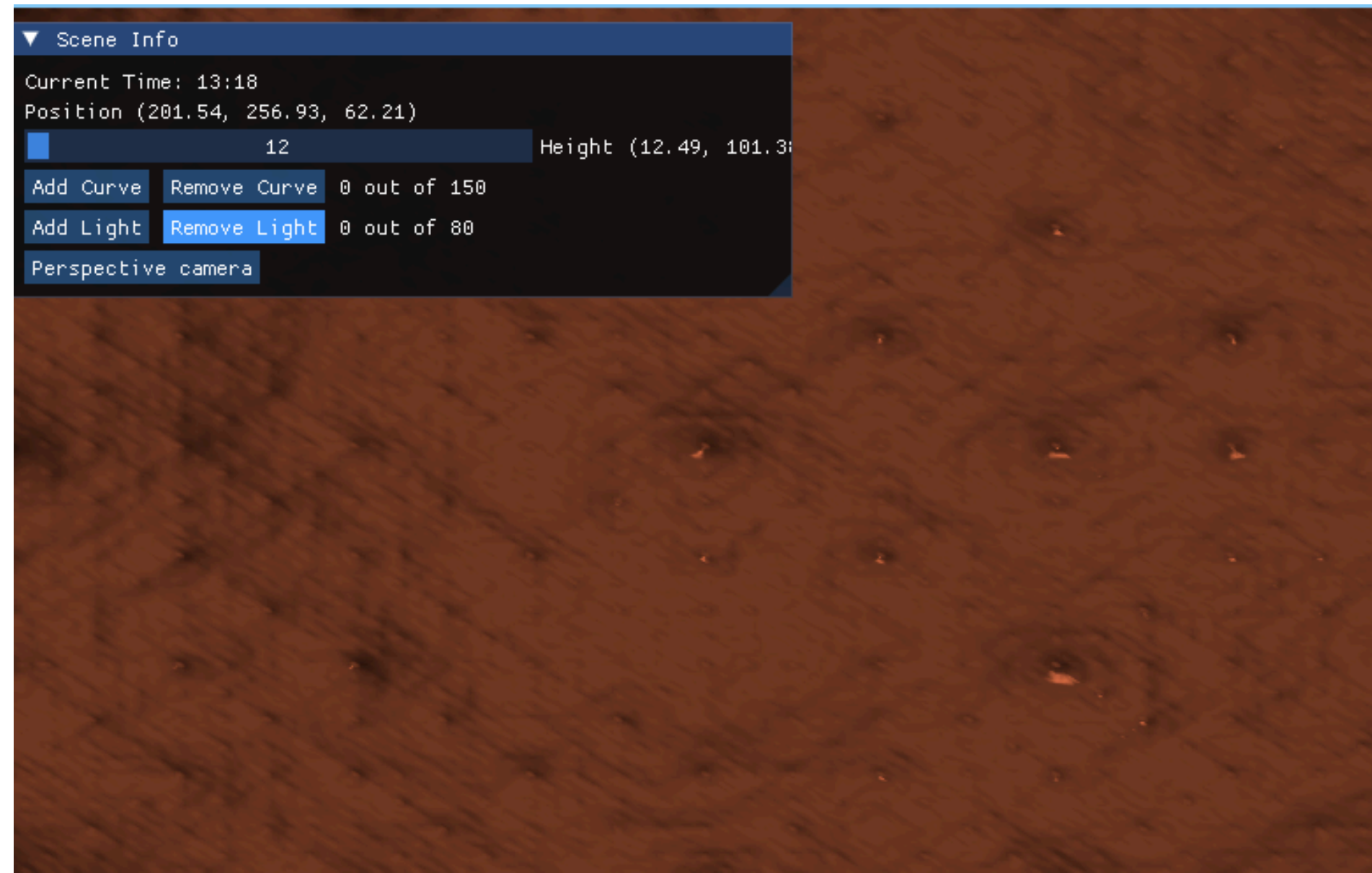
Uso de “promedio” de normales (suma de contribuciones por triángulo) representaría mejor el terreno en cada punto

Cámara de mov. libre en perspectiva

- * WASD
- * SPACE para ascender
- * SHIFT para descender
- * mouse para cambiar dirección
(cursor en bordes para rotar)
- * R para regenerar terreno



Cámara vista aérea ortogonal



Problemas con la cámara

No se recordaba el funcionamiento de las matrices de modelo, vista y proyección, y su aporte en el posicionamiento en el mundo "clipeado".

Cómo afectaba la orientación de la cámara al cálculo de las matrices y el movimiento de la misma con el mouse



Estudio, ensayo-error, repaso de ramos anteriores, búsqueda de info en librerías como glm o páginas como learnopengl.

Interfaz de usuario

Tiempo actual en la escena

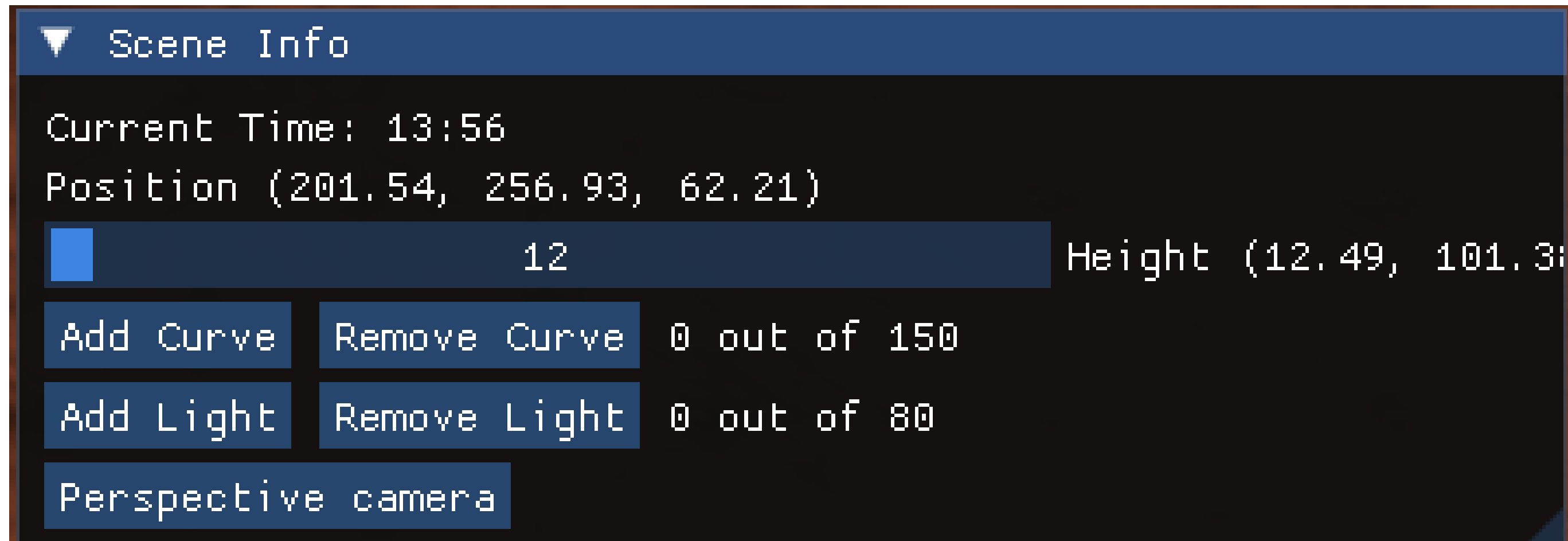
Posición de la cámara

Slider con altura para nueva curva (límites de altura entre paréntesis)

Añadir o remover curva

Añadir o remover luz puntual

Cambiar tipo de cámara



Sobre los shader

Para la generación de la malla triangular hay un pipeline con shaders:

- *Vertex shader

- *Fragment shader

```
void main() {  
    fragColor = color;  
  
    FragPos = vec3(model * vec4(position, 1.0));  
    Normal = mat3(transpose(inverse(model))) * normal;  
    gl_Position = projection * view * vec4(FragPos, 1.0f);  
}
```

El vertex shader solo prepara la posición del vértice y entrega info al fragment shader

Sobre los shader

El fragment shader se encarga de tomar los valores entregados por el vertex shader y por la cpu para colocar en escena los colores de iluminación correspondientes

```
// calculates the color when using a directional light.
vec3 CalcDirLight(DirLight light, vec3 normal, vec3 viewDir)
{
    vec3 lightDir = normalize(-light.direction);
    // diffuse shading
    float diff = max(dot(normal, lightDir), 0.0);
    // specular shading
    vec3 reflectDir = reflect(-lightDir, normal);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
    // combine results
    vec3 ambient = light.ambient * lightColor;
    vec3 diffuse = light.diffuse * diff * lightColor;
    vec3 specular = light.specular * spec * lightColor;
    return (ambient + diffuse + specular)*objectColor;
}
```

```
// calculates the color when using a point light.
vec3 CalcPointLight(PointLight light, vec3 normal, vec3 fragPos, vec3 viewDir)
{
    vec3 lightDir = normalize(light.position - fragPos);
    // diffuse shading
    float diff = max(dot(normal, lightDir), 0.0);
    // specular shading
    vec3 reflectDir = reflect(-lightDir, normal);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
    // attenuation
    float distance = length(light.position - fragPos);
    float attenuation = 1.0 / (light.constant + light.linear * distance + light);
    // combine results
    vec3 ambient = light.ambient * lightColor;
    vec3 diffuse = light.diffuse * diff * lightColor;
    vec3 specular = light.specular * spec * lightColor;
    ambient *= attenuation;
    diffuse *= attenuation;
    specular *= attenuation;
    return (ambient + diffuse + specular)*objectColor;
}
```

Sobre los shader

Para la generación de curvas de nivel hay otros shaders:

- *Vertex shader
- *Geometry shader
- *Fragment shader

El vertex shader y fragment shader solo entregan la info, no hay cálculos involucrados

```
void main() {  
    fragColor = color;  
    gl_Position = vec4(position, 1.0f);  
}
```

```
void main() {  
    outColor = vec4(fragColor, 1.0f);  
}
```

Sobre los shader

El geometry shader se encarga de lo importante: Recibe la información de las curvas a dibujar, revisa si cada primitiva está a la altura, y le dibuja un segmento perteneciente a la nueva curva.

El color de la curva es único al valor de la altura

```
void main()
{
    // Generar curvas de nivel
    for (int c = 0; c < curves; ++c) {
        float contourHeight = curve_heights[c] ;

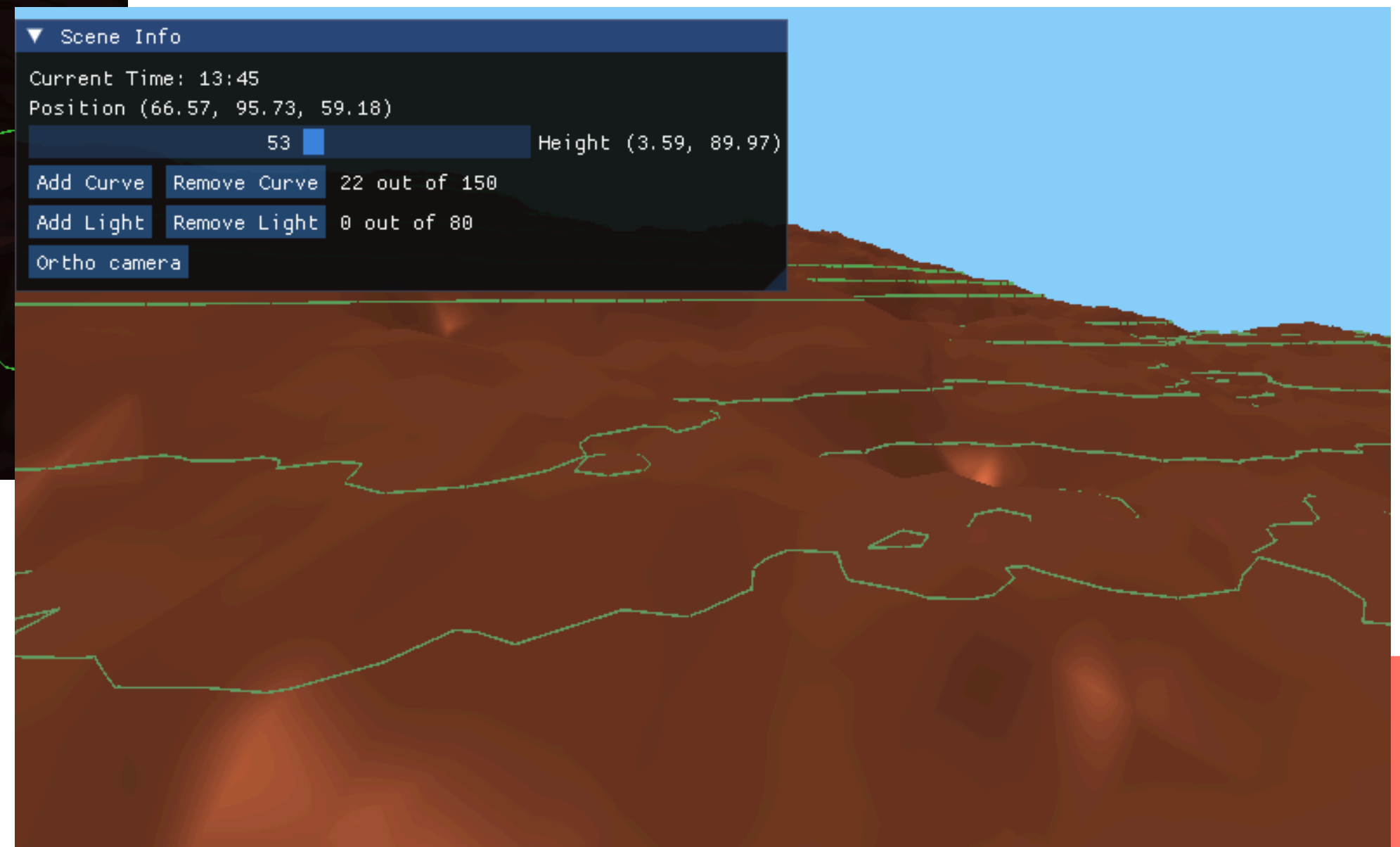
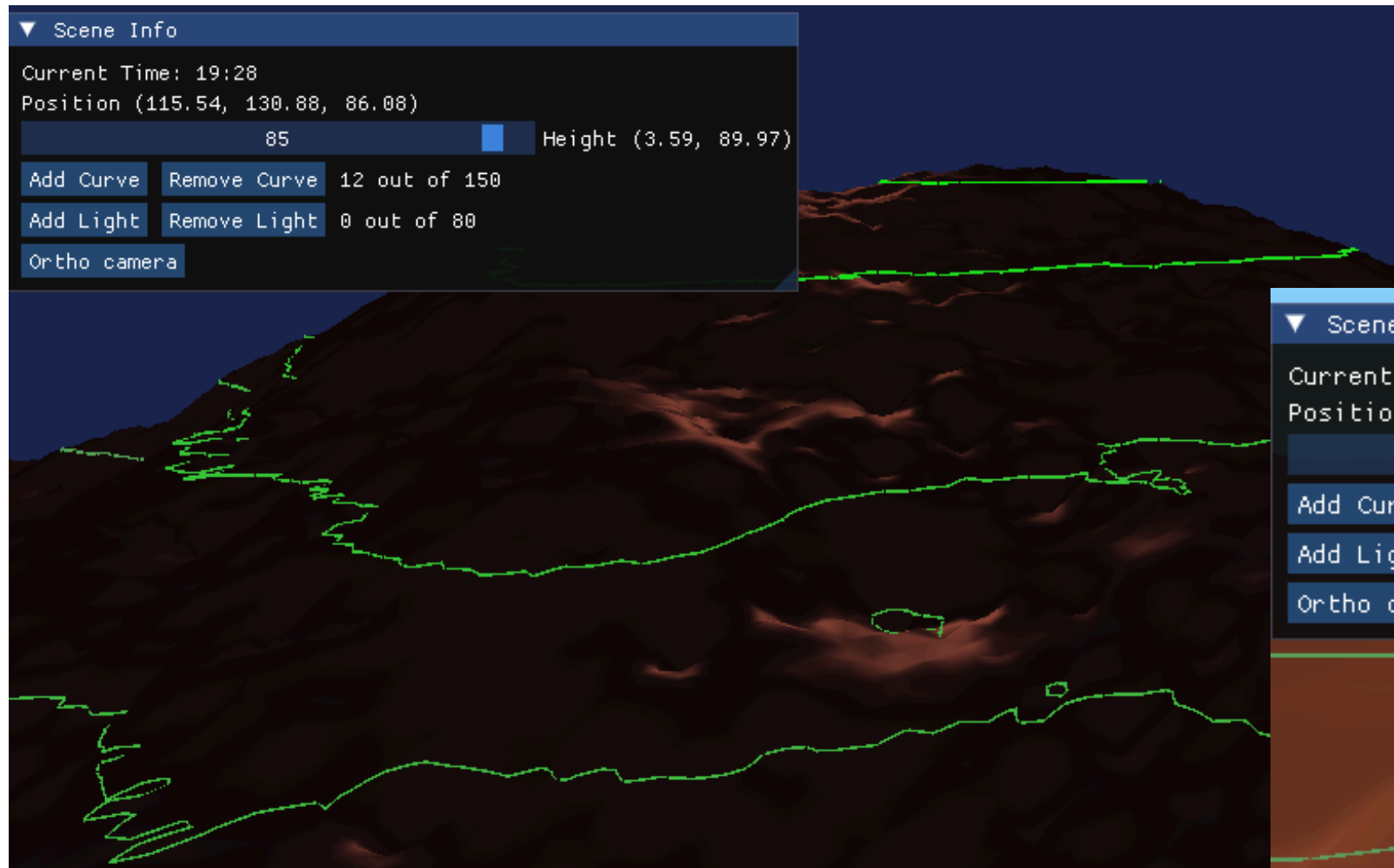
        // Emitir líneas de contorno
        for (int i = 0; i < 3; ++i) {
            int j = (i + 1) % 3;
            vec3 p0 = gl_in[i].gl_Position.xyz;
            vec3 p1 = gl_in[j].gl_Position.xyz;

            bool p0_above = p0.z > contourHeight;
            bool p1_above = p1.z > contourHeight;

            if (p0_above != p1_above) {
                float t = (contourHeight - p0.z) / (p1.z - p0.z);
                vec3 intersection = mix(p0, p1, t);
                float dtCol = (contourHeight-minHeight)/(maxHeight-minHeight);
                vec3 curveColor = vec3(1.0-dtCol, dtCol, 1.0-dtCol);
                gl_Position = projection * view * model * vec4(intersection, 1.0);
                fragColor = curveColor; // Color negro para las líneas
                EmitVertex();
            }
        }
        EndPrimitive();
    }
}
```

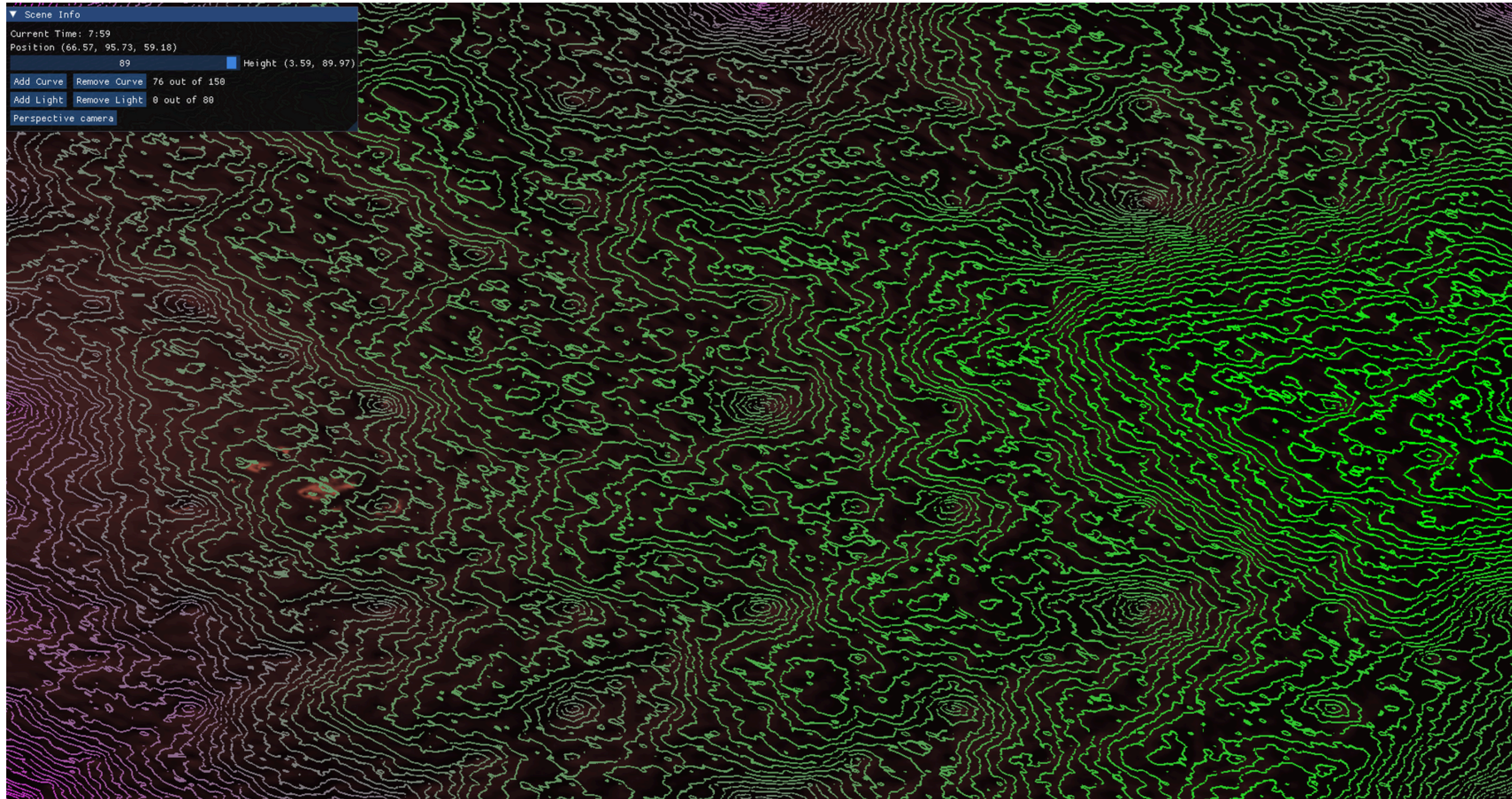
Sobre los shader

Curvas de nivel



Sobre los shader

Curvas de nivel



Sobre los shader

Se itera por todas las alturas que tendrán curva. Se revisan los 3 pares de vértices contiguos el triángulo: si uno de ellos está arriba de la altura y el otro abajo, entonces se debe dibujar un segmento, y el segmento tendrá un color único determinado por $(altura - alturaMin) / (alturaMax - alturaMin)$

La mayor complejidad se vio al tener que implementar dos conjuntos de shaders diferentes para poder visualizar las curvas al mismo tiempo que la malla

The background features a white canvas with abstract orange and red shapes. At the top, there is a horizontal orange bar and a red circle with a light red outline. At the bottom, there is another horizontal orange bar and a red circle with a light red outline. A large, light orange rounded rectangle is centered on the page, containing the text.

**Gracias por su
atención**